

# Technical Interview Task

---

## 1. Overview

This assignment evaluates practical skills in AI integration, backend engineering, and system design. The goal is to simulate a real-world scenario involving API monitoring and intelligent alert generation.

---

## 2. Task

Intelligent API Monitoring & Alert System

---

## 3. Objective

Design and implement a system that:

- Automatically monitors API responses
  - Detects anomalies
  - Generates AI-powered, human-readable alerts
- 

## 4. Scenario

The company integrates with multiple external APIs for patient and operational data. These APIs may:

- Fail intermittently
- Return inconsistent data
- Produce abnormal responses

Your task is to proactively detect such issues and automate alerting.

---

## 5. Input Methods

You may provide input to the system using any of the following:

1. Static JSON file containing API responses
2. POST requests to Node.js backend endpoint ([/monitor](#))

**Example JSON Input:**

JSON

```
[
  {
    "api_name": "PatientDataAPI",
    "response_time_ms": 1200,
    "status_code": 200,
    "records_returned": 50
  },
  {
    "api_name": "AppointmentAPI",
    "response_time_ms": 5500,
    "status_code": 500,
    "records_returned": 0
  }
]
```

---

## 6. Processing Requirements

The system must:

## 6.1 API Health Monitoring

- Evaluate response time
- Check HTTP status codes
- Validate returned data

## 6.2 Anomaly Detection

Detect conditions such as:

- High response time
- Failed requests (e.g., 500 errors)
- Missing or zero records

## 6.3 AI-Powered Alert Generation

Use an AI model (LLM API) to generate human-readable alerts.

Example Alert:

"AppointmentAPI failed with status 500 and returned 0 records. Possible outage or data issue."

---

# 7. Expected Output

## 7.1 Data Storage

- Store detected anomalies in a database (SQL or NoSQL or MongoDB)

## 7.2 API Endpoint

- Create REST endpoint: `/alerts`
- Fetch all active anomalies

## 7.3 Frontend

- Build a clean and user-friendly UI to display alerts

## 7.4 Optional Feature

- Send anomaly reports via email
- 

## 8. Technical Requirements

### Backend

- Node.js (Express recommended)
- Modular and scalable architecture

### AI Integration

- Integrate LLM API (e.g., OpenAI, Gemini, etc.)
- Generate descriptive alerts

### System Capabilities

- Handle batch processing of multiple API inputs
  - Implement proper error handling
  - Include logging system
- 

## 9. If any candidates used AI Usage Instructions

If AI is used for:

- Code generation
- Prompt design
- Logic building

You must:

- Document all prompts used
  - If any candidates used Ai then Submit them in a separate file ([AI\\_Prompts.docx](#))
- 

## 10. Evaluation Criteria

| Criteria             | Description                       |
|----------------------|-----------------------------------|
| Backend & API Design | Clean, modular, maintainable      |
| Frontend             | User-friendly UI                  |
| AI Usage             | Quality of generated alerts       |
| Data Handling        | Proper anomaly storage            |
| Automation Logic     | Correct anomaly detection         |
| Scalability          | Handles multiple APIs efficiently |

---

## 11. Submission Guidelines

### Required Deliverables:

1. Source Code
  - Upload full project to GitHub
  - Share repository link
2. README File
  - Setup instructions
  - Run instructions
3. Video Explanation (5–10 mins)
  - Explain architecture

- **Show system workflow**
  - **Highlight key decisions**
- 

## **12. Submission Format**

**Send an email containing:**

- **GitHub repository link**
  - **Cloud/Drive link (video + if any used prompts to add AI prompts file)**
- 

## **13. Deadline**

**5 days from the assignment date**

---